

Subspace Snooping: Exploiting Temporal Sharing Stability for Snoop Reduction

Jeongseob Ahn, Daehoon Kim, Jaehong Kim, and Jaehyuk Huh, *Member, IEEE*

Abstract—Although snoop-based coherence protocols provide fast cache-to-cache transfers with a simple and robust coherence mechanism, scaling the protocols has been difficult due to the overheads of broadcast snooping. In this paper, we propose a coherence filtering technique called *subspace snooping*, which stores the potential sharers of each memory page in the page table entry. By using the sharer information in the page table entry, coherence transactions for a page generate snoop requests only to the subset of nodes in the system. However, the coherence subspace of a page may evolve, as the phases of applications may change or the operating system may migrate threads to different nodes. To adjust subspaces dynamically, subspace snooping supports two different shrinking mechanisms, which remove obsolete nodes from subspaces. Among the two shrinking mechanisms, subspace snooping with safe shrinking can be integrated to any type of coherence protocols and network topologies, as it guarantees that a subspace always contains the precise sharers of a page. Speculative shrinking breaks the subspace superset property, but achieves better snoop reductions than safe shrinking. We evaluate subspace snooping with Token Coherence on unordered mesh networks. Subspace snooping reduces 58 percent of snoops on average for a set of parallel scientific and server workloads, and 87 percent for our multiprogrammed workloads.

Index Terms—Multicore/single-chip multiprocessors, cache coherence, low-power design

1 INTRODUCTION

THE two broad classes of coherence protocols, snooping protocols and directory protocols, have traditionally targeted different scales of systems. Snooping systems offer low-cost, simple coherence at small system scales. Directory protocols have been built to scale much higher, but at greater cost and complexity. In snooping protocols, there is no explicit storage, or directory, to track the sharing states of memory blocks, and coherence requests must be broadcast to all the nodes. Such broadcast snooping allows fast two-hop cache-to-cache transfers, and eliminates the complexity of maintaining the directory. However, scaling snooping protocols has been difficult due to the overheads of broadcasting requests and looking up the cache tags of all the nodes for snooping.

However, a large number of snoop requests in snooping protocols are unnecessary as the majority of memory blocks are not shared by all the nodes. There have been several recent studies to filter such unnecessary snoop requests by tracking sharing states at region granularity. Using the sharing states, the filtering techniques remove unnecessary snoop requests at requesting sources [7], [20], at receiving destinations [21], or at intermediate routers during the transmission of the requests [1]. All the techniques use some on-chip hardware tables to store the sharing states of the most recently accessed regions.

In this paper, we propose a new coherence filtering technique, called *subspace snooping*, based on the page table support of operating systems. To the best of our knowledge, this is one of the first work to use the page table to store potential sharer lists for snoop reduction. Subspace snooping filters snoops at requesting sources, reducing both snoop tag lookups at the destinations, and network traffic for transferring snoop requests. Unlike prior work based on the dichotomy of private and shared spaces [7], [20], in subspace snooping, a set of sharers defines a subspace for a page.

In the rest of this paper, a sharer of a memory region is a node which accesses the region at least once during a certain time period. A subspace is a *stable* subset of nodes sharing a region of memory consistently. These subspaces can be dynamically evolving, so long as they are stable for sufficiently long to be useful. While subspace snooping does not achieve the sharing-list precision of directory protocols, it has the potential to offer many of the benefits of snooping protocols with a reduced number of total snoops.

Subspace snooping uses an OS-based mechanism to maintain subspaces at page granularity. The set of sharers for a page (the subspace of a page) is recorded in the OS page table entry (PTE), and translation look-aside buffers (TLBs) also keep the subspace information. For a coherence transaction, requests are delivered only to the nodes in the subspace. The subspace is guaranteed to be the superset of the current precise sharers which have a copy of the requested block in their caches (*subspace superset property*). The property allows subspace snooping to be used with any type of snoop-based coherence protocols and interconnection networks.

A set of sharers at page granularity may change during program execution. Applications may have phases with

• The authors are with the Computer Science Department, Korea Advanced Institute of Science and Technology (KAIST), 373-1 Guseong-dong, Yuseong-gu, Daejeon 305-701, Republic of Korea.

E-mail: {jeongseob, daehoon, jaehong}@calab.kaist.ac.kr, jhuh@kaist.ac.kr.

Manuscript received 30 Nov. 2010; revised 7 Aug. 2011; accepted 7 Sept. 2011; published online 30 Sept. 2011.

Recommended for acceptance by L. John.

For information on obtaining reprints of this article, please send e-mail to: tc@computer.org, and reference IEEECS Log Number TC-2010-11-0660.

Digital Object Identifier no. 10.1109/TC.2011.195.

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on September 28, 2023 at 04:32:27 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Published Processor Power Breakdown

Processor	Cores	Cache	I/Os	Others
Power4	52%	23%	13%	12%
Niagara2	34%	21%	14%	31%

different sharing patterns and the operating system may migrate threads to different physical cores. Such sharer changes may decrease the effectiveness of subspace snooping, as subspaces of pages may contain the nodes which no longer access the pages.

To adjust subspaces dynamically, we investigate two subspace shrinking mechanisms, which remove obsolete nodes from the subspaces. The first mechanism, *safe shrinking*, maintains the subspace superset property, and can be used with any snoop-based coherence protocols. The second mechanism, *speculative shrinking*, can occasionally break the superset property, and thus requires the underlying coherence protocol to have support for a fallback mechanism when a sharing node does not receive a necessary snoop request. Speculative shrinking may provide better snoop reductions than safe shrinking, but imposes an extra restriction on the underlying coherence protocol.

Subspace snooping does not add a significant hardware complexity to existing snoop-based systems and is easily adaptable to novel coherence techniques such as Token Coherence and In-network Coherence Ordering [2], [17]. Compared to prior work to filter unnecessary snoop traffic, the contributions of this paper are as follows: first, unlike the region-based source filtering techniques [7], [20], subspace snooping does not simply divide the address space to private and shared regions, but can filter snoops for partially shared pages by tracking possible subsets of sharers. Second, subspace snooping requires a relatively small amount of extra hardware, as sharing information is in page tables. Third, subspace snooping is not dependent on the implementations of underlying coherence mechanisms and interconnection networks.

In this paper, we apply subspace snooping to Token Coherence, which provides simple ordering support on unordered networks, without indirection through home nodes [17]. On a 16-core system, subspace snooping reduces the total global snooping by 58 percent on average for our parallel scientific and server workloads, and 87 percent for our multiprogrammed workloads, compared to the base TokenB protocol. Filtering snoop requests at requesting sources reduces network traffic and power consumption significantly. The network traffic is reduced by 36 percent for the parallel workloads, and 51 percent for the multiprogrammed workloads.

In the rest of this paper, we first discuss the limitation of snooping protocols, and prior work on snoop filtering in Section 2. In Section 3, we present the sharing characteristics of applications at page unit. In Section 4, we describe the subspace snooping architecture. In Section 5, we examine the effects of sharing changes and thread migrations, and we introduce two shrinking mechanisms to mitigate the effects. In Section 6, we discuss the costs of implementing

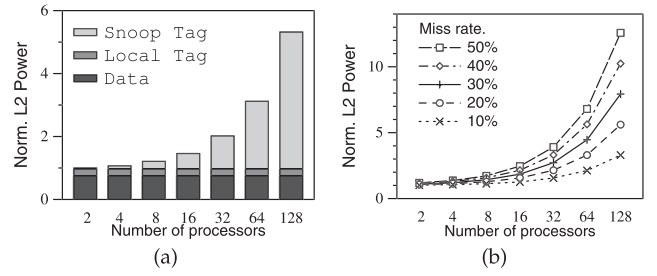


Fig. 1. Dynamic power consumption of L2 caches.

subspace snooping. Section 7 presents experimental results, and Section 8 concludes the paper.

2 BACKGROUND

2.1 The Limitations of Snoop-Based Coherence

Snooping coherence requires high bandwidth for networks to transfer snoop requests and responses, and enough access bandwidth for snooping cache tags. With the improvement of inexpensive on-chip networks, the available network bandwidth has been increasing, but the bandwidth of snoop tag must be increased too, as the snoops tags of all nodes must be accessed for every coherence transaction. Furthermore, in addition to the high bandwidth requirement for networks and cache tags, their power consumption must also be considered to improve the scalability of snoop-based protocols.

Table 1 shows the power breakdown of Power4 and Niagara2 [22] processor. Caches account for 21-23 percent of the total processor power. Note that the ratios of cache power consumption are from a single-node or small-scale system. However, as the number of nodes increases, a prior study showed that a significant fraction of L2 cache power is consumed for snoop tag lookups [21]. With a simple analytical model, we demonstrate the extent to which the power consumption of snooping tags will increase with the number of nodes.

Fig. 1 shows L2 dynamic power consumption for each node with increasing numbers of nodes. Each node has a private L2 cache connected by a snooping coherence protocol. In Fig. 1a, the per-node dynamic L2 power is further divided for data accesses, tag lookups by the local core (local tag), and snoop tag lookups. L2 cache miss rates are 10 percent for all the numbers of nodes. Although the power consumption for snoop tag lookups accounts for a very small portion of the total L2 power for two nodes, the portion increases rapidly with more nodes. With 64 nodes, snoop tag lookups consume more than 60 percent of the total dynamic power of an L2 cache. An L2 cache in a 64-node system consumes three times more power than an L2 in a two-node system due to the increase of power consumption for snoop tags. Fig. 1b presents per-L2 dynamic power with different L2 cache miss rates. As the cache misses increase, the overhead of snoop tag lookups increases significantly.

Network power consumption will contribute to a significant portion of the total chip power, as the number of snoops increases. According to Vangal et al., up to 28 percent of total chip power is consumed by the interconnection

networks [28]. In Alpha 21364, the integrated router and links account for about 20 percent of the total chip power.¹ The power consumption by networks will increase significantly as there will be more broadcasting snoops with increasing core counts.

2.2 Snoop Filtering Techniques

There are several studies to reduce unnecessary snoops in snoop-based coherence, and subspace snooping is based on the prior work on snoop filtering techniques. *RegionScout* and *Coarse-Grain Coherence Tracking* (CGCT) filter snoop requests for private data at requesting sources. RegionScout records the states (private or shared) of coarse-grained regions in per-node tables [20]. CGCT also maintains an additional coarse-grained coherence protocol in addition to the conventional coherence at cacheline granularity [7]. The two techniques store the coarse-grained coherence states in hardware tables, and support only the dichotomy of coherence states into private or shared states to reduce the area overhead. Even if only two nodes share a region of data, coherence transactions on the data must be broadcast to all the nodes.

Subspace snooping differs from RegionScout and CGCT in that subspace snooping does not simply divide coherence space into private or shared states. By storing sharer information in page tables, subspace snooping can select any possible subset of sharing nodes for each page.

Unlike the aforementioned source-level filtering techniques, *In-Network Coherence Filtering* (INCF) filters out snoop requests at the routers [1]. Each router maintains a table for region-based sharing states, and does not forward a snoop request to a port, if the snoop request does not need to be delivered to the nodes reachable from the port. To support such filtering, the routing algorithm must be deterministic and does not allow flexible adaptive routing. Unlike INCF, subspace snooping does not require any modification to underlying networks. *Flexible snooping* proposed by Strauss et al. provides adaptive forwarding and filtering of snoop requests, either for high performance or energy conservation [25].

Jetty filters out unnecessary snoops at destination nodes, by maintaining a filter in each node [21]. Since the filter resides at the destination node, snoop requests must still be broadcast to all the nodes, consuming network bandwidth and power. It saves only the power consumption to look up snoop tags. *RegionTracker* proposes a mechanism to track cache states at both fine- and coarse-grain granularity with a two-level dual-grain tracking mechanism [30].

Ekman et al. embedded sharing vectors in the TLBs with virtually addressed caches [9]. The page-level sharing vector in the TLBs is used to filter out unnecessary snoop requests. Subspace snooping keeps the subspace information in the TLBs, but the original subspace information is stored in the page table. Also, subspace snooping does not track the sharing vector for every coherence transaction, which may require a constant update of the sharing vector in the TLBs. Virtual snooping exploits memory isolation among virtual machines to prevent snoop requests from crossing the virtual machine boundary [14].

1. Alpha 21364 designers estimate the integrated router and links to dissipate 25 W out of the total chip power of 125 W, with the router core consuming 7.6 W, link circuitry (drivers, pad logic) consuming 13.3 W, clocks taking up 2 W, and miscellaneous circuitry the remaining.

This paper extends our prior study with a new shrinking mechanism, the analysis and evaluation of multiple address spaces, and the consideration for scalability, clustering, and superpaging [13].

2.3 Other Related Work

Subspace snooping is extensively influenced by the prior work to improve coherence bandwidth, to support snooping on unordered networks, and to use OS support for managing on-chip caches.

2.3.1 Improving Coherence Bandwidth

Subspace snooping contains elements of both traditional snooping and directory protocols to improve coherence bandwidth. Bandwidth Adaptive Snooping has combined the benefits of snooping and directories adaptively in one system [19]. Multicast snooping [5] and Destination Set Prediction [16] use prediction techniques which send coherence requests only to potential sharers. Thus, these schemes must support a fall-back mechanism to recover when a miss prediction occurs.

2.3.2 Coherence Ordering on Unordered Networks

There are several recent studies to embed coherence capabilities into on-chip networks. Token coherence replaces the conventional cache states with tokens to remove synchronous updates of cache states [17], [24]. In-network cache coherence proposes the embedding of cache coherence protocols within the network [8]. The protocol sends a request to the nodes in a virtual tree that consists of the sharers. Virtual tree coherence supports virtually ordered tree interconnects on unordered networks, and multicasts to the sharers tracked by region [10].

2.3.3 OS-Based Cache Management

There are recent studies to use OS supports to manage on-chip cache placements. Fensch and Cintra proposed an OS-based approach for cache coherence in tiled CMPs by maintaining a single copy for each address [11]. Reactive-NUCA classifies data into private data, instruction, and shared data. Blocks are placed to a location that can improve performance by the classification [12]. In this approach, they record classification information in page tables.

3 FINDING SUBSPACES

Subspace snooping uses stable subspaces found in the sharing behaviors of parallel applications. To analyze the sharing behaviors, we present the distributions of sharers at page granularity, and show that the majority of page accesses occur for partially shared pages. We use the Simics full-system simulator [15] with a cache model in the GEMS toolset [18]. The target system has 16 cores with a 512 KB private L2 cache per core. We use parallel applications from SPLASH-2 [29] and PARSEC [4], and two server applications, SPECjbb and Apache. We also evaluate three multi-programmed applications, mix1, mix2, and mix3. The details of the benchmark applications and system configuration are shown in Tables 4 and 5.

Subspaces may contain more cores than the precise sharers for two reasons: 1) *spatial* and 2) *temporal* aliasing

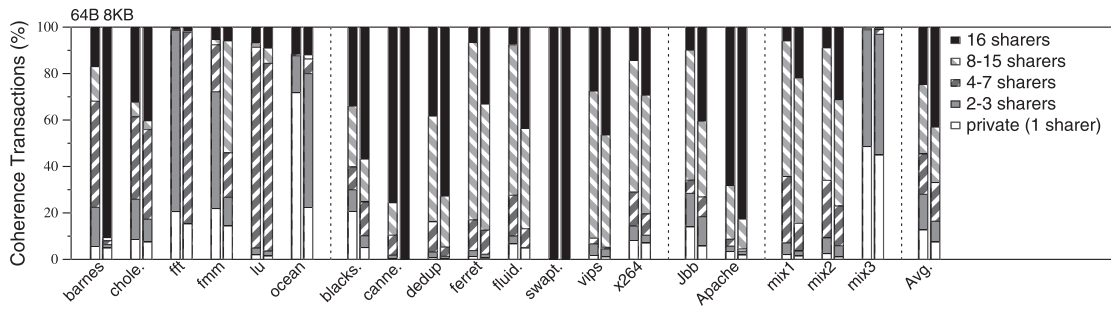


Fig. 2. Sharing distributions of coherence transactions: 64 B and 8 KB granularity (16 cores).

effects. Spatial aliasing occurs, since subspaces are maintained at page granularity. The chance of false sharing with page granularity is higher than that with cacheline granularity. Second, temporal aliasing occurs, since subspace snooping does not update sharers for each coherence transaction as directory protocols do. Therefore, subspaces may have sharing cores which have accessed a page a while ago, but no longer have any cachelines of the page in their caches.

3.1 Sharing Behaviors at Page Granularity

Subspace snooping tracks sharers at page granularity and sends snoop requests only to the sharers marked in the page table. To reduce snoop requests, the number of sharing nodes for each page must be small. Fig. 2 presents sharing distributions for all coherence transactions, which include read, read-exclusive, and upgrade (from shared to modified) transactions in the MOESI protocol. For each page, we track sharers cumulatively from the beginning to the end of simulation. In the figure, the number of sharers for a coherence transaction is the number of sharers collected for the accessed page until the transaction occurs. Sharers tracked in each page are never removed during execution. For each application, the figure shows two distributions, the first one with 64 B granularity and the second one with 8 KB granularity. The distributions of sharers are divided into 1 (private), 2-3, 4-7, 8-15, and 16 core partitions.

As shown in Fig. 2, applications have significant potentials to reduce snoops, even at page granularity. On average, 7.4 percent of coherence requests are for private pages, 9.0 percent for 2-3 sharers, 16.4 percent for 4-7 sharers, 33.1 percent for 8-15 sharers, and 56.9 percent for 16 sharers. Canneal and swaptions in PARSEC show the worst sharing behaviors, with almost the entire coherence transactions sent to fully shared pages. Commercial workloads, SPECjbb and Apache, access fully shared pages at higher rates than the average rate.

The figure also shows that, for snoop requests from L2 caches, 50 percent of coherence requests occur on partially shared pages with 2-15 sharers. This result indicates that dividing the address space only to private and shared spaces does not provide an effective snoop reduction. With such dichotomy, 93 percent of accesses occur on shared pages, and snoop requests must be broadcast.

Tracking sharers at page granularity causes spatial aliasing for some applications. The coherence transactions on fully shared pages in barnes increase from 17 (block granularity) to 90.6 percent (page granularity). Dedup also shows a significant increase of fully shared page accesses

from 64 B to 8 KB granularity (from 38.3 to 72.8 percent). The rest of the applications show modest increases of sharers with page granularity. Compiler or programming optimization to reduce false sharing at page granularity may be able to reduce the spatial aliasing, but the techniques are beyond the scope of this paper.

3.2 Multiple Address Spaces

Multiprogrammed workloads where each application runs in a separate address space can provide an ideal case for subspace snooping. In current shared-memory systems, even though many processes are running on the systems without any sharing, all the memory operations by the processes must be checked for coherence, causing unnecessary snoop requests. Subspace snooping can eliminate such unnecessary snooping across separate address spaces.

Mix1, 2, and 3 in Fig. 2 present the sharing distributions of multiprogrammed workloads. Mix3 consists of applications with one or two threads, and coherence requests for private pages accounts for over 50 percent in mix3. In addition, there is no significant difference between page and block granularity. For mix1 and mix2 including applications with four to six threads, the portions of private page are relatively small, but partially shared pages dominate the sharing patterns. For the two mixes, there are significant accesses for 8-15 sharers, even though the maximum number of threads is six. It is mainly due to the migration of threads to different cores by the operating system. Subspace snooping tracks physical cores, and thus a thread migration results in an addition of a new physical core to the subspace. In Section 5, we will investigate the effect of thread migrations on subspace snooping.

4 SUBSPACE SNOOPING ARCHITECTURE

Subspace snooping maintains potential sharer lists in page table entries and TLBs. To describe the sharer tracking mechanism of subspace snooping, we first present bispace snooping, which divides subspaces only to two spaces, private and shared. Bispace snooping is similar to the prior coarse-grained source-level filtering, except for the use of OS page tables. In Section 4.2, we introduce a general subspace snooping protocol to track all the possible combinations of sharers.

4.1 Bispace Snooping Architecture

As a simplified subspace snooping, we introduce *bispace snooping* that records the private or shared states of pages in page table entries. Before cores send coherence requests, the cores must look up the TLBs for address translation. With

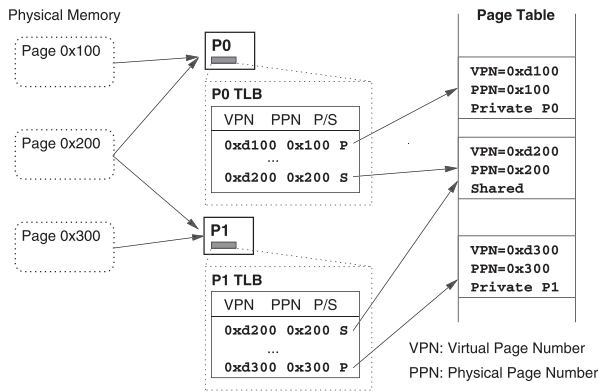


Fig. 3. Bispace snooping using a page table and TLBs.

bispace snooping, the cores can find whether pages are shared or private during address translation. For private pages, snoop requests do not need to be sent to the other cores. In conventional snooping protocols, if a cache miss occurs, the requesting core does not know whether the cacheline is shared or private until all the other cores are snooped. Therefore, even if a page is private (no other cores have ever accessed the page), a core must broadcast requests and all the other cores must snoop to verify whether the cacheline is in their caches.

Using page tables and address translation mechanisms, bispace snooping classifies pages into private and shared pages to choose a different space. If a page has been accessed by only one core, the page is set as a private page for the core. If another core attempts to access the page, the page state is updated to a shared state. For private pages, bispace snooping does not broadcast requests, reducing unnecessary snoops.

Fig. 3 shows the overview of a bispace snooping protocol using a page table. To classify pages, bispace snooping adds an extra bit in page table entries for representing a private or shared state. TLB entries must also have the extra state bit. In addition to the state bit, a page table entry maintains an owner identifier for private pages, and the owner identifier of a page is set when a core accesses the page table entry for the first time to handle a TLB miss. Bispace snooping updates the page sharing state in a page table entry when the TLB miss handler accesses the page table entry to fill TLBs. If a page is a private page already owned by another core, the sharing state of the page should be changed to a shared state.

When a page state changes from private to shared, a critical constraint for correctness is that the original owner must also be notified about the change, before a new sharer accesses the memory location. To do that, the TLB miss handler must send a TLB update request to the current owner to update the status of its TLB entry to shared. To avoid any race condition, the TLB miss handler must not complete the TLB fill until the TLB entry of the current owner is completely updated. To ensure the correctness, the TLB miss handling of the newly joining core is delayed until the acknowledgment from the current owner is received.

4.2 Fine-Grained Subspace Snooping

Subspace snooping extends bispace snooping to support fine-grained subspaces for each page. Fig. 4 presents the overview of subspace snooping. The sharing states in the

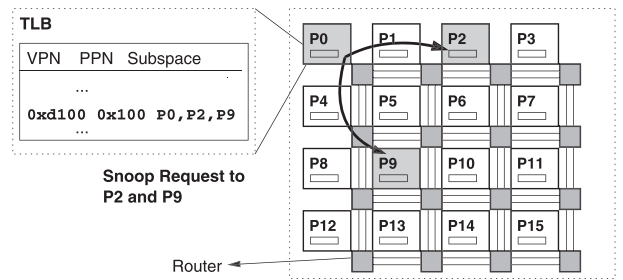


Fig. 4. Subspace snooping overview: sending a snoop request to a subspace.

TLBs and page table are extended to sharing vectors to record the sharers of each page. For each coherence transaction, a requesting core finds a set of sharers (subspace) of the address from the TLBs. Snoop requests are delivered only to the cores in the subspace. In Fig. 4, the subspace of page 0x100 is {P0, P2, P9}, and P0 sends snoop requests only to P2 and P9.

To maintain the list of sharers for each page, page table entries and TLBs are extended to hold a bitmap, *sharing vector* for all the cores, with each bit corresponding to each core. For every coherence transaction, the requesting core looks up the TLBs during address translation, and sends requests to only the cores marked in the sharing vector.

Subspace snooping uses a similar mechanism to bispace snooping to keep the TLBs up to date with the latest subspace information. Whenever a new core is added to a subspace, the sharing vector of the page table entry and the TLB entries of cores in the current subspace must be updated. To avoid any race condition, the update must be completed, before the new core is permitted to access the page. In bispace snooping, sharing state updates happen only at transitions from private to shared. However, in subspace snooping, a subspace is updated whenever a new core is added to the subspace.

The generalized subspace snooping may incur more storage and timing overheads than bispace snooping. First, subspace snooping must update the subspace of a page more frequently than bispace snooping, since adding each new sharer must update the subspace. In the worst case, one page table entry can be updated N times when N is the number of cores. Such updates delay the TLB miss handling of a new sharer joining the subspace. Second, with subspace snooping, the page table size may be increased in the main memory, and the TLBs require more space for sharing vectors than that with bispace snooping. We will discuss the costs of supporting subspace snooping in Section 6.1.

5 SUBSPACE POLLUTION

Subspace pollution occurs when a subspace contains obsolete sharers, which accessed the page before, but no longer access it recently. As the numbers of obsolete sharers increase in subspaces, subspace snooping will become less effective. In this section, we present two reasons for subspace pollution, and propose two mechanisms to adjust subspaces dynamically.

5.1 Factors Causing Subspace Pollution

The subspace of a page changes over the runtime of an application for two reasons. First, sharing patterns among

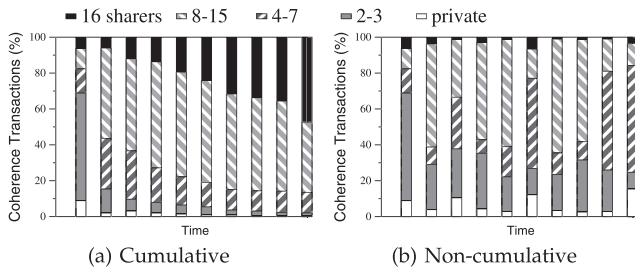


Fig. 5. Sharing distributions for different time intervals.

threads in the application change dynamically. Second, the physical cores a thread is running on may change by the operating system. Subspace snooping tracks physical cores, not logical threads. If a thread migrates to a new physical core, subspace snooping will add the new physical core to the subspace when the page is accessed from the new core. In parallel applications, the combined effects cause subspace pollution, but in single-threaded applications, only the migration effect will cause subspace pollution.

Fig. 5 shows cumulative and noncumulative sharing distributions at different time intervals. In the cumulative distribution of Fig. 5a, sharers are cumulatively added to subspaces from the beginning of each run, and are never removed. In the noncumulative distribution of Fig. 5b, sharer lists are reset at the beginning of each interval. In the figures, we show the patterns of a representative application, *ferret*. The differences between two distributions show the effect of obsolete sharers building up in subspaces during the runtime. In the cumulative distribution, there are high ratios of 8-15 sharer and fully shared pages, but in the noncumulative distribution, less cores are sharing pages with very low ratios of fully shared page accesses. Also, the distributions at different time intervals fluctuate significantly in the noncumulative distribution, showing program phase changes.

A mapping change between a thread and a physical core affects subspace changes, even if there is only one thread in each application. Operating systems migrate threads to different cores to maximize the overall system throughput or balance the load on each core. If a thread migrates from a core to a new core, the base subspace snooping adds the new core to the subspace of a page, as soon as the thread starts accessing the page from the new core.

Fig. 6 shows a migration history for selected two threads in *dedup*. Y-axis of the figure is the physical core a thread is running on. In applications with dynamic parallelism, such thread migrations may occur frequently. For example, applications may be parallelized with a thread pool, and the tasks are assigned to threads dynamically. Some threads in the pool may be actively running, and the others may not, which cause unbalanced loads on physical cores. In such applications, frequent thread migrations by scheduling changes cause subspace pollution. In *PARSEC*, *dedup*, *ferret*, and *x264* use such a parallelization model, and adversely affect subspace snooping. Also, frequent I/O activities can cause thread migrations, as threads enter and exit sleep states frequently, waiting for I/O responses.

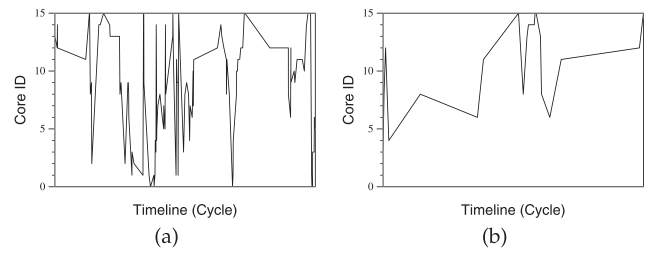


Fig. 6. Thread migrations of two different threads.

5.2 Safe Shrinking

To adapt to sharing phase changes or thread migrations, subspace snooping can support two mechanisms to remove sharers from subspaces dynamically. The first mechanism, called *safe shrinking*, uses a conservative policy which allows it to be used with any coherence protocols. In the next section, we present a more aggressive mechanism, called *speculative shrinking* with potentially better snoop reductions, but with some restrictions on the underlying protocol.

Shrinking subspace requires the coordinated updates of page table entries and TLBs in other cores. To shrink a subspace, a core, which attempts to remove itself from the subspace, updates the sharing vector in the page table entry, and sends subspace shrink requests to the current sharers to update their TLBs. However, updating the TLBs of other nodes can be delayed, without causing any correctness problem. As long as the sharing vectors cached in the TLBs are the supersets of a subspace in the page table, subspace snooping works correctly.

The safe shrinking mechanism tracks the number of blocks of a page residing in the caches of each node. When a page is completely evicted from a node, the node is removed from the subspace of the page.

When subspaces are shrunk, one critical invariant must be maintained for correctness:

- *Subspace invariant*. If a node contains a cacheline in its caches or a page in its TLBs, the node must be in the subspace of the page including the cacheline or the corresponding TLB entry.

Safe subspace shrinking maintains the subspace invariant, and thus supports the subspace superset property guaranteeing that a subspace is always the superset of precise sharers of all the cachelines in the page. To trigger a safe shrink, the mechanism must know when the last cacheline of a page is evicted from the local caches. When no cacheline of a page exists in the caches, a node can remove itself safely without violating the subspace invariant. The corresponding TLB entry must also be invalidated.

Supporting the mechanism requires a fast page-level cache residence test function to check whether a page is cached or not. For the cache residence test, we use a counting bloom filter technique [6]. Each node has a bloom filter indexed by a hash of a physical page number (PPN). The bloom filter entry has a counter and a virtual page number (VPN). Whenever a cacheline is inserted to the local caches, the counter in the corresponding entry is increased by 1. When a cacheline is evicted or invalidated, the counter is decreased by 1. A problem with the bloom filter is the

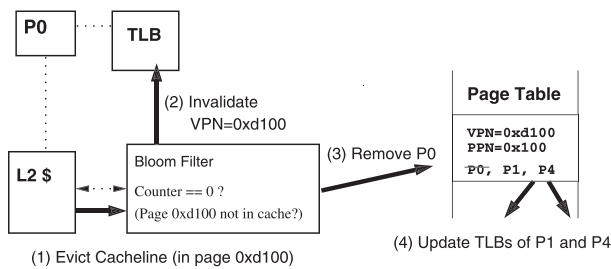


Fig. 7. Safe shrinking: removing P0 for page 0xd100.

saturation of counters, which should be a rare event. If the counter of an entry is saturated, we assume that all PPNs mapped to the entry have cachelines in the local caches. Infrequently, the OS can reset the bloom filter, when the entire local caches are flushed. If an aliasing, with two pages mapped to the same entry, occurs, only one of the pages can be shrunk, due to one VPN per entry. In this paper, the bloom filter uses 1,024 entries for each core, with 8 bits for each entry.

Fig. 7 describes the shrinking processes. A bloom filter is used to trigger shrink events. When a counter is decreased and becomes zero, it is guaranteed that pages mapped to the entry no longer exist in the caches. Using the stored VPN, the local TLB is invalidated and the page table is updated.

5.3 Speculative Shrinking

Safe shrinking can remove a core from the subspace of a page, only when no cache block of the page exists in the core. In this section, we propose speculative shrinking mechanisms, which relax such a strict restriction. Speculative shrinking aggressively removes a potentially obsolete core from a subspace, even if such removing may violate the superset property. It can be used only with a coherence protocol, which supports a fall-back mechanism for misspeculations. Token coherence is one of such protocols which allow speculative coherence transactions. In token coherence, if necessary tokens are not collected, the transaction can be safely retried until all the required tokens are collected. In token coherence with the subspace shrinking support, if the first attempt for a coherence transaction fails due to incomplete subspace information, the request can be retried by broadcasting to all the nodes.

In this paper, we propose two speculative shrinking schemes. The first scheme exploits the property of read for sharing requests, which need to be sent only to the owner of the block. The second scheme addresses subspace pollutions caused by thread migrations.

5.3.1 Optimizing for Read Requests

For read for sharing requests, which do not invalidate the copies of the cache block in the other cores, coherence messages do not need to be sent to all the cores with the cache blocks. In directory-based protocols, such a request can be served by the home node, if the home node memory has the valid copy, or the request is forwarded to a node with a modified copy.

Similarly, for snoop-based coherence protocols, a request to read for sharing can be delivered only to the core responsible for providing the cache block (the owner of the

block). The owner will provide the requested data whereas other sharers, even if they have copies, may ignore the request. Even if a nonowner sharer does not receive a read request to the block, due to a miss speculation, it does not violate the coherence of the block. The ownership of the block can be marked in different ways by different protocols. In a typical MOESI protocol, a node with a modified (M) or owned (O) state block has the ownership for the block. Some protocols may designate one of the nodes with shared (S) state blocks as the owner of the block, if there are no M or O state blocks in the other caches. The owner in shared state is responsible for providing the data by cache-to-cache transfers.

In token coherence, a read request must be sent to the owner node of a block, which has available tokens for the block. The owner sends the data along with a token to the requesting node. If the read request is not sent to the owner, the requesting node will receive neither the data nor a token, so it will retry the same transaction, but broadcast requests for the second try. For a write request, the request must be delivered to the nodes caching the block, to collect all existing tokens. If the requester cannot collect all the tokens for the block, it will retry by broadcasting requests.

The first speculative shrinking mechanism tracks the number of owner blocks of a page residing in the caches of each node, instead of blocks in any state in safe subspace shrinking. The mechanism is similar to the safe shrinking mechanism with a per-core bloom filter, but the counter for each slot is increased or decreased only for blocks with ownership states. When a counter becomes zero, the node is removed from the corresponding subspace. As the bloom filter counts only the blocks with ownership states, a core can be removed from the subspace of a page, even if the core may have cache blocks with nonownership states.

5.3.2 Addressing Thread Migrations

The second speculative shrinking mechanism augments the first mechanism by addressing thread migrations. A thread newly scheduled to a core commonly does not use the data cached by the previous thread. The old data in the cache are gradually evicted from the cache, as the new thread brings its own working set to the cache. When a thread leaves a physical core by a scheduling change, the core is still included in the subspaces of pages accessed by the thread, as subspace snooping tracks physical cores in subspaces.

The safe shrinking mechanism waits for the evictions of all the cache blocks used by the previous thread to shrink the subspaces for the cache blocks. This second speculative shrinking mechanism removes a core much sooner than the safe shrinking mechanism, if a thread leaves the core.

To track the thread migration, each core maintains an epoch identifier. An *epoch* is the time period from the scheduling of a thread to a core until the thread leaves the core. The per-core epoch identifier is increased for every thread change. For each bloom entry used for the first speculative shrinking, in addition to the counter for owner blocks, we add a field for epoch ID. The epoch ID field is updated to the current epoch whenever the core accesses any block corresponding to the entry. To reduce the storage overhead for epoch IDs, we use only one bit for epoch. The epoch of a core switches between zero and one for each

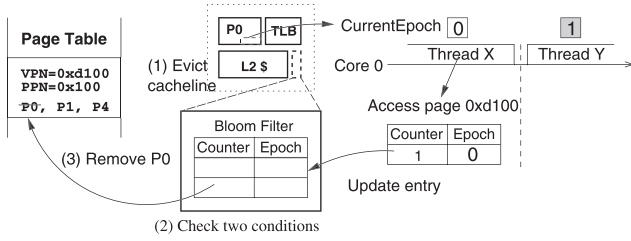


Fig. 8. Speculative shrinking: removing P0 for page 0x100.

thread change. The reason we use only one bit for epoch is that the cached blocks by the previous thread are commonly evicted completely while the current thread is running.

Each bloom filter entry records whether the page has been used by the current thread. To combine the epoch-based scheme to the first scheme for read for sharing requests, we use the following condition to trigger a speculative subspace shrinking for an eviction of a block with an ownership state. OwnerCounter and Epoch are from the bloom filter entry of the evicted block, and CurrentEpoch is the current epoch ID of the core.

- (OwnerCounter == 0) OR
(Epoch != CurrentEpoch AND OwnerCounter < Threshold)

When a block with an ownership state is evicted from the cache, the mechanism first checks whether the counter reaches zero. If the condition is met, the core is removed from the subspace of the page. The second condition checks whether the page is being used by the current thread, and if not, the condition checks whether the counter is less than a threshold. If the page has not been used by the current thread, the mechanism shrinks the subspace even before the counter reaches zero. In this paper, we use 6 as the threshold.

Fig. 8 describes the speculative shrinking processes. A bloom filter and epoch are used to trigger speculative shrinking events. When a counter is decreased and becomes a zero, it satisfies the first condition as mentioned above. Although a counter is nonzero, when the epoch of the entry differs from the current epoch of the node, it satisfies the second condition.

6 MAINTAINING SUBSPACES

6.1 Overheads in TLBs and Page Tables

6.1.1 Available Space in PTE

To record subspaces in page tables, a page table entry must have enough free space. As discussed in [3], UltraSPARC-III supports 64-bit address space, but uses only 44 bits for virtual addresses and 41 bits for physical addresses. The remaining 23 bits are unused and those bits can be used for storing subspace information. If the number of nodes exceeds 23, a possible solution is to cluster the nodes. 32-core machine can use 16 bits, with clustering two cores to a cluster. Sharing vectors are set for cluster units, not for individual cores. If a requesting core sends snoop requests to a cluster, all cores in the cluster must snoop the request.

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on September 28, 2023 at 04:32:27 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Access Latencies for 64-Entry TLBs (45 nm)

	4-way		FA	
PTE size	64-bit	128-bit	64-bit	128-bit
latency (ns)	0.3542	0.4101	0.5026	0.5183

6.1.2 TLB Access Latencies

If unused bits in TLB entries are not available, TLB entries must be increased to store sharing information. Such increase of TLB entries may affect the access latencies of TLBs. To quantify the effect on TLB latencies, we estimate the access latencies of TLBs with CACTI [26]. Table 2 shows the TLB access latencies with 64-bit and 128-bit entry sizes. We evaluate two possible organizations of 64-entry TLBs, four-way associative and fully associative (FA) TLBs. We use 64 TLB entries to model level-1 TLBs used in commercial processors, since the effect of TLB latencies is critical for L1 TLBs. In the table, even if the size of each TLB entry doubles, the latency increases are relatively small. For the full associative configuration, the latency increase is negligible. For the four-way associative TLB, the latency increases by 16 percent, but the difference is still small, which takes the same one clock cycle with 2 GHz clock frequency.

6.2 TLB Consistency for Subspace Changes

In conventional shared-memory multiprocessors, a page table entry is updated when the virtual-to-physical mapping is changed or the permission status is changed. In subspace snooping, changing subspaces also requires updating the page table. Such page table updates require a mechanism to keep TLBs consistent [27]. To reduce the overheads of maintaining TLB consistency for subspace changes, subspace snooping uses a logic component in each TLB to update subspace bits by requests from other cores. When a subspace is updated for a page, the update requests for the TLBs are multicast to the cores in the current subspace. The receiving cores must send acknowledgments to the requesting core, after updating the corresponding bit in their TLBs.

For a normal TLB miss, the page table entry is fetched either from caches or the external memory. If a subspace addition is necessary, an atomic read-modify-write is performed to the entry which is already in the local cache by the TLB fill. For SW-based TLB fills, the TLB fill handler will update the page table entry. For HW-based TLB fills, the TLB fill controller must be augmented for the support.

Unlike the page table updates to change address mapping or permission bits, adding a core to a page table entry does not result in a significant complexity for avoiding a race condition. Each core changes only its corresponding bit in the sharing vector, but it never modifies the other bits in the page table entry. However, it needs an atomic read-modify-write operation to set the bit safely without data races.

After the page table entry is updated, the requesting core broadcasts subspace update messages to all the cores in the current subspace. The requesting core can access the page only after it receives the acknowledgments from all the other cores. Compared to a normal TLB fill process, a subspace add

TABLE 3
Subspace Updates per 1,000 Instructions

Workloads	base	safe shrink		spec. shrink	
	add	add	remove	add	remove
barnes	0.002	0.003	0.001	0.007	0.005
cholesky	0.006	0.033	0.031	0.062	0.058
fft	0.039	0.071	0.055	0.084	0.069
fmm	0.002	0.002	0.002	0.004	0.004
lu	0.002	0.002	0.002	0.005	0.004
ocean	0.004	0.056	0.054	0.096	0.095
blackscholes	0.007	0.009	0.002	0.012	0.006
canneal	0.053	0.206	0.161	0.225	0.180
dedup	0.014	0.029	0.016	0.139	0.136
ferret	0.004	0.008	0.006	0.010	0.008
fluidanimate	0.010	0.018	0.010	0.026	0.021
swpations	0.001	0.032	0.031	0.086	0.085
vips	0.004	0.015	0.013	0.026	0.025
x264	0.001	0.003	0.002	0.003	0.003
SPECjbb	0.091	0.126	0.056	0.234	0.168
apache	0.103	0.356	0.277	0.960	0.909
mix1	0.008	0.019	0.013	0.036	0.033
mix2	0.008	0.027	0.022	0.034	0.031
mix3	0.003	0.020	0.018	0.021	0.020

operation requires two extra steps: 1) first, it needs the execution of an atomic read-modify-write to set the corresponding bit in the page table entry. The operation may require a shared to modified state change for the cache block of the page table entry. The cacheline for the entry will be most likely in the local caches, since the TLB fill operation brings it into the caches. 2) The second step is to send TLB update messages to the other cores in the subspace, and wait for the acknowledgments. The latencies of each step are close to the latencies of broadcasting cacheline invalidation requests and receiving the acknowledgments.

Shrinking subspace can increase subspace updates, as it can remove a core prematurely from a subspace, incurring subspace additions later. Unlike adding a new core to a subspace, removing a core from a subspace during shrinking is not on the critical path of TLB or cache miss handling. Shrinking can be processed in the background, as delayed shrinking only increases unnecessary snoops.

6.3 Subspace Update Rates in Applications

Table 3 presents the number of subspace updates per 1,000 instructions. The second column shows the number of subspace updates to add a new core with the base subspace protocol. The base subspace snooping has a minor number of subspace updates. FFT has 0.039 updates per 1k instructions, and canneal, SPECjbb, and apache have 0.053, 0.091, and 0.103 updates, respectively. The rest of the applications have less than 0.01 updates per 1k instructions. The result shows that subspace updates are very infrequent. The third and fourth columns of Table 3 show the number of updates for adding and removing a core with the safe shrinking support. An observation is that with safe shrinking, compared to the base subspace, the subspace add rates are increased almost by the same amount as the subspace shrink rates. It means most of the shrunken pages are accessed again later by the removed cores. The benchmark applications do not have many coherence requests on the pages with obsolete sharers in the subspace, making the safe shrinking mechanism ineffective. With speculative shrinking, the subspace add

TABLE 4
Simulated System Configurations

Parameter	Value
Processors	16 in-order SPARC core
L1 I/D cache	32KB, 4-way, 64B block
L2 cache	512KB, 8-way, 64B block
I-TLB	L1 : 16-entry, fully assoc. L2 : 128-entry, 2-way
D-TLB	L1 : 16-entry, fully assoc. L2 : 2 × 512-entry, 2-way
Main memory	4GB memory, 8KB pages
Bloom filter size	1024 entries, 7 bit counter 1KB per core
Coherence	Token Coherence
On-chip Network	4x4, 2D mesh with 16B links
Orion	4 cycle router pipeline Tech. 65nm, Freq. 1GHz

rates are slightly increased for canneal and apache. For the rest of applications, the subspace add rates do not change significantly, compared to safe shrinking.

6.4 Sharing Physical Pages

If the virtual pages of different address spaces are mapped to the same physical page, subspace snooping may not track the sharers correctly. A solution for correctness is to simply force broadcast snoops for the aliased pages, overriding subspace snooping. As the OS is always aware of such sharing and able to mark it on the page tables, subspace snooping does not cause any correctness problem. Direct Memory Access (DMA) to cacheable memory by I/O devices must broadcast the requests bypassing the subspace snooping mechanism. However, such DMA accesses are not frequent compared to other coherence transactions.

7 EXPERIMENTAL RESULTS

To evaluate subspace snooping, we use Virtutech Simics [15], a full-system simulator, with a timing model from the GEMS toolset [18]. The GEMS memory model is augmented with the GARNET interconnection model [23]. Our parallel workloads consist of applications from SPLASH-2 [29], PARSEC [4], SPECjbb2000, and Apache. Apache is a webserver workload with the Apache HTTP server, serving only static webpages. To evaluate applications with multiple address spaces, we use three multiprogrammed workloads, which consist of PARSEC and SPEC CPU2006. The details of workload parameters are shown in Table 5.

The simulated system is a 16-core CMP and each core is an in-order SPARC processor with a private L2. Table 4 shows the configurations of the simulated system. We model 4 × 4 2D-mesh interconnection networks, with dimension ordered routing. We integrate subspace snooping to Token Coherence (TokenB), including the impact of supporting TLB coherence for subspace changes. Token coherence uses persistent requests as a fall-back mechanism, if normal transactions exceed the time-out limit. We do not apply subspace snooping to the persistent requests to simplify our integration with TokenB. For speculative shrinking, if the first trial of a coherence transaction fails due to the lack of necessary tokens, the second trial broadcasts requests overriding the request filtering by the subspace information.

The per-core bloom filter for shrinking has 1,024 entries and

TABLE 5
Application Input Data and Parameters

Parallel Workloads			
Application	Dataset	Application	Dataset
barnes	64K particles	fmm	64K particles
cholesky	tk29.0	lu	1024 × 1024
fft	4M points	ocean	514 × 514 grid
PARSEC	Simmedium		
SPECjbb2K	16 warehouses, 10000 trans.		
Apache	160 simulated users, 4000 trans.		

Multi-programmed Workloads	
mix1	vips(4), x264(4), ferret(4), dedup(4)
mix2	ferret(6), vips(4), xalancbmk, mcf, gcc, povray, bzip, hmmer
mix3	bzip(2), gcc(2), mcf(2), xalancbmk(2) hmmer(4), povray(2), swpations, fluidanimate

each entry has a 7-bit counter and 1 bit for epoch. Using the TokenB protocol as the baseline, we evaluate six configurations, bspace, base subspace snooping without shrinking (subspace in figures), safe shrinking, speculative shrinking without epoch, speculative shrinking with epoch, and an ideal configuration (ideal).

7.1 Snoop Reduction

In this section, we present the snoop reductions by various subspace snooping schemes. We measure how many snoops occurring in all the nodes can be reduced by subspace snooping. Fig. 9 shows the reduction of total number of snoops occurring at all the cores, compared to the baseline TokenB protocol, which always broadcasts snoop requests. The last bar of each application shows the snoop reduction with an ideal protocol. In the ideal protocol, a requesting core always knows exactly which cores have a copy of the cache block requested for the coherence transaction, which is similar to a directory-based protocol. The requesting core can send requests only to the precise sharers. The ideal protocol sends read for sharing requests only to the owner core, even if multiple cores may have the cache block.

Bspace snooping has modest snoop reductions in several applications. Mix3 has the highest reduction with bspace (57 percent), followed by fft with 50 percent. Cholesky, fmm, lu, and ocean have 24-29 percent snoop reductions by removing snoop requests for private pages. However, the rest of the applications have small reduction rates with bspace snooping. Although the memory overhead of bspace snooping is very small, its benefit is modest or minor.

Using fine-grained subspaces improves bspace significantly. For the parallel workloads, the base subspace snooping without shrinking reduces snoops by 40 percent

on average, reducing 25 percent more snoops compared to the 15 percent reduction of bspace. The snoop reduction rates for SPLASH-2 workloads are generally higher than those of PARSEC and commercial workloads. Three applications, fft, lu, and ocean, are close to the ideal since the sharing patterns of these workloads have most of the coherence transactions on 1-7 sharer pages. For the multi-programmed workloads (mix1, 2, and 3), snoop reductions are very high, 67 percent on average, as expected.

Two applications with the worst reduction rates are canneal and swaptions. Canneal has very fine-grained sharing patterns, where threads access small data items in a random pattern. In such fine-grained random sharing patterns, subspace snooping cannot form stable subspaces with small numbers of sharers. However, the ideal protocol can reduce the snoops by 94 percent for canneal. Canneal shows the inability of subspace snooping to track precise sharers, if many cores access shared pages frequently, but only a small subset of the cores keep copies of the pages in their caches.

Swaptions also shows that most of the coherence transactions occur on the pages shared by all the cores, as shown in Fig. 2. Swaptions has little communication among threads, but the majority of misses occur for instruction fetching, as the instruction working set does not fit in the private L2. Since the memory pages for instructions are shared by all the cores, when a cache miss occurs for a core, most likely other cores have the cacheline in their caches.

The third bar of Fig. 9 shows the snoop reduction rate by the safe subspace shrinking mechanism for each application. The results show that the safe shrinking mechanism can improve the reduction rates to 48 percent, and 76 percent for the parallel and multiprogrammed workloads, respectively. The benefit of safe shrinking is small in the benchmarks we used. The safe shrinking scheme, which conservatively waits for the entire eviction of a page from a node, cannot identify obsolete sharers fast enough to reflect sharing changes and thread migrations.

Speculative subspace shrinking improves the base subspace snooping significantly by filtering snoops for read for sharing requests, and by adjusting subspace speculatively for thread migrations. Between the two speculative schemes, the optimization for read transactions contributes to more reductions than that for thread migrations. For the parallel workloads, speculative shrinking with both the owner counter and epoch reduces snoops by 55 percent on average, reducing 15 percent more snoops compared to the base subspace snooping without shrinking. Especially, the relatively long-running workloads such as vips, ferret, and x264 show significant improvements of snoop reduction rates. In SPLASH-2 [29], all the threads are statically

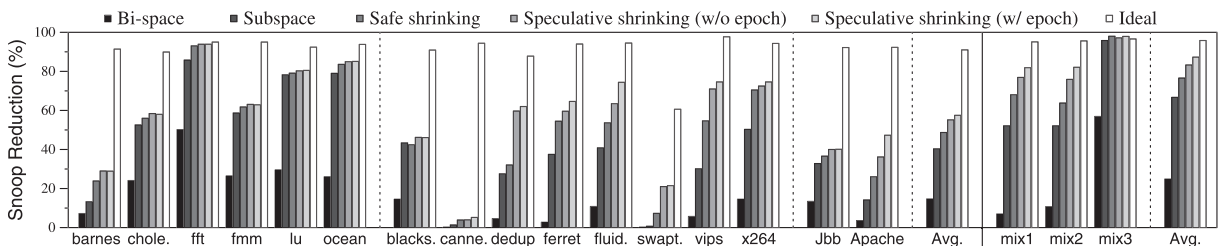


Fig. 9. Snoop reduction with Token Coherence: 16 cores (parallel and multiprogrammed workloads).

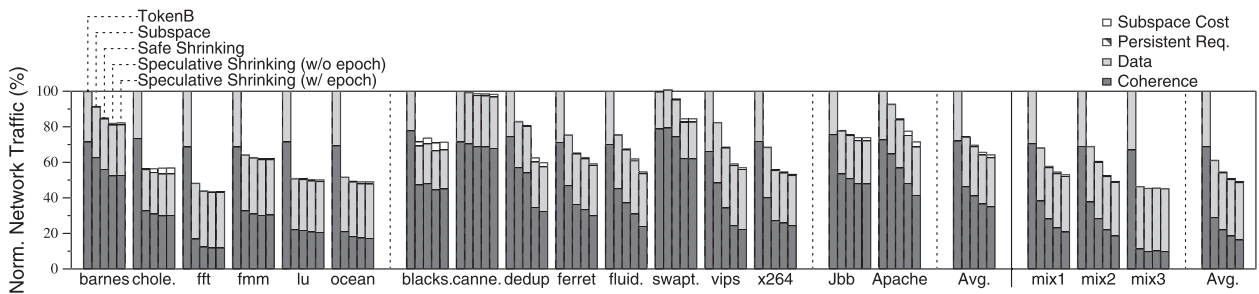


Fig. 10. Normalized network traffic: 16 cores (parallel and multiprogrammed workloads).

created for all physical cores with a one-to-one mapping, and the mapping changes rarely during the runtime. In such applications, speculative shrinking does not improve the safe shrinking scheme significantly. The multiprogrammed workloads also exhibit more reductions with the speculative shrinking mechanism, with 87 percent reduction on average.

7.2 Network Traffic and Performance

In this section, we present the reduction of network traffic by filtering snoop requests on a 4×4 mesh network, and performance improvement by the traffic reduction.

7.2.1 Network Traffic

By reducing snoop requests at requesting cores, subspace snooping reduces the network traffic in token coherence. Fig. 10 shows the network traffic with subspace snooping normalized to that with the base token coherence. The five bars for each application show the network traffic with TokenB, subspace, safe shrinking, and two speculative shrinking schemes, respectively. For each bar, the traffic is divided into coherence, data, persistent and subspace update requests. For the parallel workloads, with TokenB, 72 percent of the total traffic is for coherence requests and acknowledgments, and 28 percent is for data traffic. The traffic by persistent requests is negligible. Subspace snooping reduces only the coherence traffic. The multiprogrammed workloads show a similar decomposition of traffic for TokenB.

With subspace snooping, *fft*, *lu*, and *ocean* have about 50 percent less network traffic than TokenB. In *canneal* and *swaptions*, network traffic does not decrease, as requests are mostly to fully shared pages. The average network traffic is reduced by 36 and 51 percent for the parallel and multiprogrammed workloads with speculative shrinking mechanisms. Except for *blackscholes*, most of workloads have negligible traffic overheads for subspace update messages, since subspaces are updated infrequently. However, even in *blackscholes*, the subspace update messages increase the traffic only by 2 percent.

The power consumption by networks shows a similar trend to the network traffic rates. On average, the network power can be reduced by 24 percent without shrinking, and by 37 percent with speculative shrinking from TokenB. The power reduction including snoop tag lookups leads to a significant on-chip power saving for future many cores.

7.2.2 Performance Improvement

Fig. 12 shows the normalized execution times of three configurations. For *cholesky*, *fft*, and *ocean*, subspace reduces the execution times by 7-8 percent. The average execution times are reduced by 4.3 and 3.2 percent for the parallel and multiprogrammed workloads, respectively. The performance improvement is modest as the benchmark applications do not consume the network bandwidth intensively, and the network provides enough bandwidth even for TokenB. However, in this paper, we did not explore the performance improvement by other benefits of subspace snooping. For example, the reduced power consumption in the networks and snoop tag lookups allows the power budget to be used for better performance. The processor clock speed can be increased using the saved power budget, or power-limited performance features can be used more aggressively using the budget. The traffic reduction can also allow narrow network links. The saved area with the narrow links can be reused to increase cache or other prediction table sizes.

7.3 Scalability

In this section, we present the scalability of subspace snooping with increasing core counts. The scalability of subspace snooping represents whether subspace snooping can sustain snoop reduction rates even if the number of cores increases. Such scalability depends on how the sharing behaviors change with increasing core counts. To show the changing effectiveness of subspace snooping with different numbers of cores, Fig. 11 presents snoop reduction rates with increasing numbers of cores from four to 64 cores. For a core count, each bar shows a snoop reduction rate by

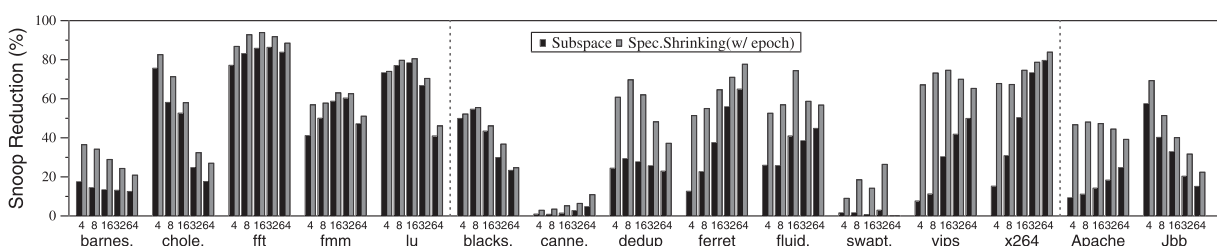


Fig. 11. Scalability: each bar shows the snoop reduction from TokenB with the same number of core.

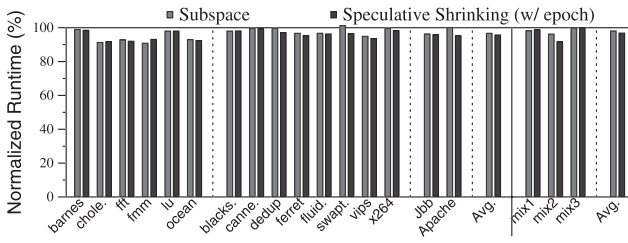


Fig. 12. Normalized execution times: 16 cores.

subspace snooping with no shrinking or speculative shrinking from the broadcasting token coherence. Note that a reduction rate for each core count is from the comparison against the TokenB with the same core count, and the absolute amount of snoop reduction increases from four to 64 cores, as the total snoops with TokenB increase.

In three applications, *cholesky*, *blackscholes*, and *jbb*, the effectiveness of subspace snooping does not scale with the number of cores, decreasing snoop reduction rates with more cores. For *cholesky* and *blackscholes*, spatial aliasing effects become more severe with increasing core counts. With 64 cores, transactions to fully shared pages increase from 29 percent at block granularity to 68 percent at page granularity for *blackscholes*, and from 26 to 79 percent for *cholesky*. The increases of fully shared accesses from block to page granularity are much higher with 64 cores than those with 16 cores shown in Fig. 2. For *SPECjbb*, even the snoop reduction by the ideal configuration also decreases with more cores, as the memory pages tend to be shared by more sharers, if the number of cores increases.

Except for the three applications, snoop reduction rates do not change significantly even if the number of cores increases. Two applications, *ferret* and *x264*, show the increases of reduction rates with more cores, showing the improved effectiveness of subspace snooping with more cores.

7.4 The Effects of Superpaging and Clustering

7.4.1 Large Page

Increasing page size may adversely affect subspace snooping, since the effect of spatial aliasing may increase with large pages. Fig. 13 presents the snoop reduction rates with

8 and 64 KB pages for subspace snooping with no shrinking, safe shrinking, or speculative shrinking. The effect of increasing the page size to 64 KB is minor for most of the applications. Without shrinking, the snoop reduction rate decreases by 9 percent on average for the parallel workloads, and 7 percent for the multiprogrammed workloads. The decreases of snoop reduction are high for some applications with the base subspace snooping without shrinking (*fft*, and *PARSEC* applications). However, with the shrinking mechanism, the adverse effect of spatial aliasing becomes minor. Superpaging with 64 KB page size affects subspace snooping with marginal degradations of snoop reductions, decreasing snoop reductions by 5 percent for the parallel workloads with speculative shrinking.

7.4.2 Clustering

With increasing number of cores, the available space in a page table entry may be not enough to assign one bit for each core. If the page table entry does not have enough bits for all the cores, a solution is to cluster the neighboring cores, making each bit represent a cluster. Fig. 14 shows the effect of clustering two or four cores to a cluster in the 64-core configuration. The figure compares 1:1, 2:1, and 4:1 clustering cases, which use 64, 32, or 16 bits to accommodate 64 cores. Although the snoop reduction rates of subspace decrease by clustering, the effects are relatively small. For the parallel applications in the 64-core configuration, clustering two and four cores decreases the average snoop reduction by 5 and 12 percent, respectively, for subspace snooping.

7.5 Spatial and Temporal Aliasing

To isolate the effects of spatial and temporal aliasing on subspace snooping, Fig. 15 presents the decomposition of total snoops into five categories. *Base* is the snoop which can be eliminated by the base subspace snooping without any shrinking techniques. *Spatial* is the unnecessary snoops caused by spatial aliasing, and can be eliminated if the base subspace snooping is used at 64 B cacheline granularity. *Temporal* aliasing is decomposed into two classes. First, *temporal (pin)* shows the snoops which can be eliminated by pinning threads to physical cores, isolating the effect of thread migration. Second, *temporal (other)* presents the

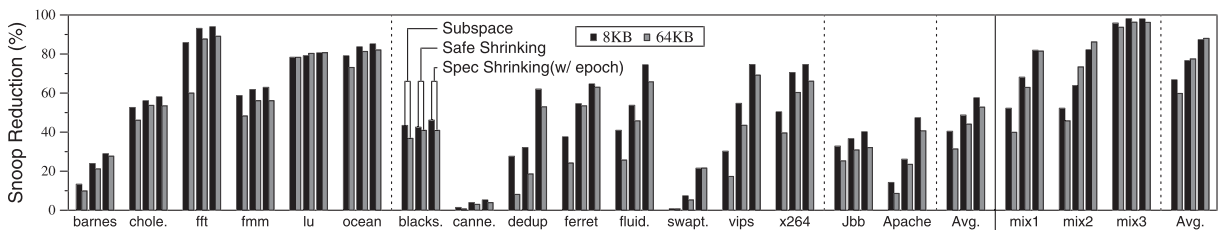


Fig. 13. Effect of the large page: 16 cores.

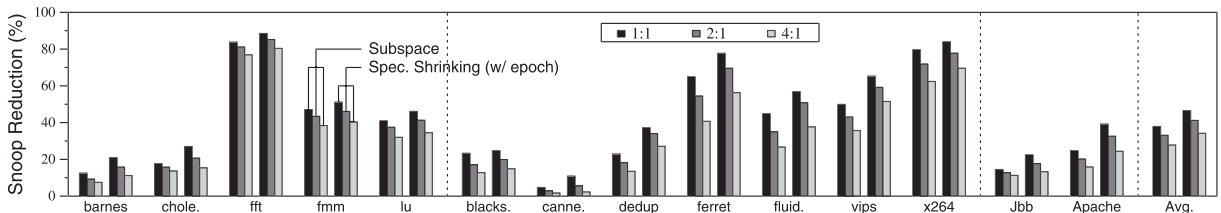


Fig. 14. Effect of clustering for 64 cores: 1:1, 2:1, and 4:1 clusters.

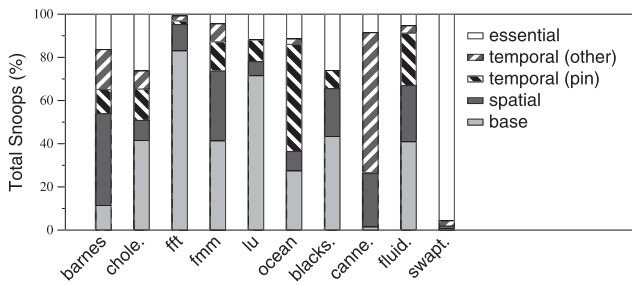


Fig. 15. Decomposition of snoops by spatial and temporal aliasing.

snoops caused by the other temporal aliasing effects. Essential is the snoop which cannot be reduced even if the spatial and temporal aliasing effects are eliminated. Note that the ideal protocol used in Section 7 can further reduce snoops in the essential category by sending at most one snoop request for each shared read transaction.

The effect of spatial aliasing is high for a subset of the applications. For *barnes*, spatial aliasing causes 42.5 percent of the total snoops. *fmm*, *blackscholes*, *canneal*, and *fluidanimate* have 22-32 percent of the total snoops in the spatial aliasing category. Temporal aliasing also causes a significant amount of snoops for *ocean* and *canneal*, with 52 and 65 percent of the total snoops, respectively. For *ocean*, the majority of spatial aliasing can be eliminated with pinning. However, in *canneal*, pinning is not effective at all, and the other temporal aliasing effects account for the most of snoops (65 percent). As shown in the results in Fig. 9, shrinking techniques mitigate the negative impact of spatial and temporal aliasing.

8 CONCLUSIONS

In this paper, we proposed a novel coherence filtering technique called subspace snooping. Subspace snooping maintains the temporally stable sharers (subspace) of a memory location in the page table entry, and snoop requests are issued only to the cores in the subspace. By using OS-page tables, the proposed technique requires a modest modification of HW designs with a small area overhead. Furthermore, the changes in the operating system are minor, as only the TLB handling mechanism needs to be updated.

ACKNOWLEDGMENTS

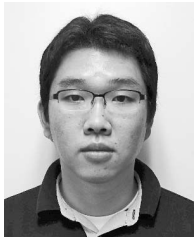
This research was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education, Science and Technology (2011-0026465).

REFERENCES

- [1] N. Agarwal, L.-S. Peh, and N.K. Jha, "In-Network Coherence Filtering: Snoopy Coherence without Broadcasts," *Proc. 42nd Ann. IEEE/ACM Int'l Symp. Microarchitecture (MICRO)*, pp. 232-243, 2009.
- [2] N. Agarwal, L.-S. Peh, and N.K. Jha, "In-Network Snoop Ordering (INSO) : Snoopy Coherence on Unordered Interconnects," *Proc. 15th Int'l Symp. High Performance Computer Architecture (HPCA)*, pp. 67-78, Feb. 2009.
- [3] M. Awasthi, K. Sudan, R. Balasubramanian, and J. Carter, "Dynamic Hardware-Assisted Software-Controlled Page Placement to Manage Capacity Allocation and Sharing within Large Caches," *Proc. 15th Int'l Symp. High Performance Computer Architecture (HPCA)*, pp. 250-261, Feb. 2009.

- [4] C. Bienia, S. Kumar, J.P. Singh, and K. Li, "The PARSEC Benchmark Suite: Characterization and Architectural Implications," *Proc. 17th Int'l Conf. Parallel Architectures and Compilation Techniques (PACT)*, pp. 72-81, 2008.
- [5] E.E. Bilir, R.M. Dickson, Y. Hu, M. Plakal, D.J. Sorin, M.D. Hill, and D.A. Wood, "Multicast Snooping: A New Coherence Method Using a Multicast Address Network," *Proc. 26th Ann. Int'l Symp. Computer Architecture (ISCA)*, pp. 294-304, 1999.
- [6] B.H. Bloom, "Space/Time Trade-Offs in Hash Coding with Allowable Errors," *Comm. ACM*, vol. 13, no. 7, pp. 422-426, 1970.
- [7] J.F. Cantin, M.H. Lipasti, and J.E. Smith, "Improving Multiprocessor Performance with Coarse-Grain Coherence Tracking," *Proc. 32nd Ann. Int'l Symp. Computer Architecture (ISCA)*, pp. 246-257, 2005.
- [8] N. Easley, L.-S. Peh, and L. Shang, "In-Network Cache Coherence," *Proc. 39th Ann. IEEE/ACM Int'l Symp. Microarchitecture (MICRO)*, pp. 321-332, 2006.
- [9] M. Ekman, P. Stenström, and F. Dahlgren, "TLB and Snoop Energy-Reduction Using Virtual Caches in Low-Power Chip-Multiprocessors," *Proc. Int'l Symp. Low Power Electronics and Design (ISLPED)*, pp. 243-246, 2002.
- [10] N.D. Enright Jerger, L.-S. Peh, and M.H. Lipasti, "Virtual Tree Coherence: Leveraging Regions and In-Network Multicast Trees for Scalable Cache Coherence," *Proc. 41st IEEE/ACM Int'l Symp. Microarchitecture (MICRO)*, pp. 35-46, 2008.
- [11] C. Fensch and M. Cintra, "An OS-Based Alternative to Full Hardware Coherence on Tiled CMPs," *Proc. 14th Int'l Conf. High Performance Computer Architecture (HPCA)*, pp. 355-366, Feb. 2008.
- [12] N. Hardavellas, M. Ferdman, B. Falsafi, and A. Ailamaki, "Reactive NUCA: Near-Optimal Block Placement and Replication in Distributed Caches," *Proc. 36th Ann. Int'l Symp. Computer Architecture (ISCA)*, pp. 184-195, 2009.
- [13] D. Kim, J. Ahn, J. Kim, and J. Huh, "Subspace Snooping: Filtering Snoops with Operating System Support," *Proc. 19th Int'l Conf. Parallel Architectures and Compilation Techniques (PACT)*, pp. 111-122, 2010.
- [14] D. Kim, H. Kim, and J. Huh, "Virtual Snooping: Filtering Snoops in Virtualized Multi-Cores," *Proc. 43rd Ann. IEEE/ACM Int'l Symp. Microarchitecture (MICRO)*, pp. 459-470, 2010.
- [15] P. Magnusson, M. Christensson, J. Eskilson, D. Forsgren, G. Hallberg, J. Hogberg, F. Larsson, A. Moestedt, and B. Werner, "Simics: A Full System Simulation Platform," *Computer*, vol. 35, no. 2, pp. 50-58, Feb. 2002.
- [16] M.M.K. Martin, P.J. Harper, D.J. Sorin, M.D. Hill, and D.A. Wood, "Using Destination-Set Prediction to Improve the Latency/Bandwidth Tradeoff in Shared Memory Multiprocessors," *Proc. 30th Int'l Symp. Computer Architecture (ISCA)*, pp. 206-217, 2003.
- [17] M.M.K. Martin, M.D. Hill, and D.A. Wood, "Token Coherence: Decoupling Performance and Correctness," *Proc. 30th Int'l Symp. Computer Architecture (ISCA)*, pp. 182-193, June 2003.
- [18] M.M.K. Martin, D.J. Sorin, B.M. Beckmann, M.R. Marty, M. Xu, A.R. Alameldeen, K.E. Moore, M.D. Hill, and D.A. Wood, "Multifacet's General Execution-Driven Multiprocessor Simulator GEMS Toolset," *SIGARCH Computer Architecture News*, vol. 33, no. 4, pp. 92-99, 2005.
- [19] M.M.K. Martin, D.J. Sorin, M.D. Hill, and D.A. Wood, "Bandwidth Adaptive Snooping," *Proc. Eighth Int'l Symp. High Performance Computer Architecture (HPCA)*, pp. 251-262, Feb. 2002.
- [20] A. Moshovos, "RegionScout: Exploiting Coarse Grain Sharing in Snoop-Based Coherence," *Proc. 32nd Int'l Symp. Computer Architecture (ISCA)*, pp. 234-245, 2005.
- [21] A. Moshovos, G. Memik, B. Falsafi, and A.N. Choudhary, "JETTY: Filtering Snoops for Reduced Energy Consumption in SMP Servers," *Proc. Seventh Int'l Symp. High Performance Computer Architecture (HPCA)*, pp. 85-96, 2001.
- [22] U. Nawathe, M. Hassan, K. Yen, A. Kumar, A. Ramachandran, and D. Greenhill, "Implementation of an 8-Core, 64-Thread, Power-Efficient SPARC Server on a Chip," *IEEE J. Solid-State Circuits*, vol. 43, no. 1, pp. 6-20, Jan. 2008.
- [23] L.-S. Peh, N. Agarwal, N. Jha, and T. Krishna, "GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator," *Proc. Int'l Symp. Performance Analysis of Systems and Software (ISPASS)*, pp. 33-42, Apr. 2009.
- [24] A. Raghavan, C. Blundell, and M.M.K. Martin, "Token Tenure: Patching Token Counting Using Directory-Based Cache Coherence," *Proc. 41st IEEE/ACM Int'l Symp. Microarchitecture (MICRO)*, pp. 47-58, 2008.

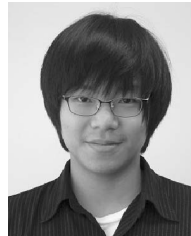
- [25] K. Strauss, X. Shen, and J. Torrellas, "Flexible Snooping: Adaptive Forwarding and Filtering of Snoops in Embedded-Ring Multiprocessors," *Proc. 33rd Ann. Int'l Symp. Computer Architecture (ISCA)*, pp. 327-338, 2006.
- [26] D. Tarjan, S. Thoziyoor, and N. Jouppi, "CACTI 4.0," technical report, HP Labs, 2006.
- [27] P.J. Teller, "Translation-Lookaside Buffer Consistency," *Computer*, vol. 23, no. 6, pp. 26-36, June 1990.
- [28] S. Vangal, J. Howard, G. Ruhl, S. Dighe, H. Wilson, J. Tschanz, D. Finan, P. Iyer, A. Singh, T. Jacob, S. Jain, S. Venkataraman, Y. Hoskote, and N. Borkar, "An 80-Tile 1.28TFLOPS Network-on-Chip in 65nm CMOS," *Proc. IEEE Int'l Solid State-State Circuits Conf.*, pp. 98-589, Feb. 2007.
- [29] S.C. Woo, M. Ohara, E. Torrie, J.P. Singh, and A. Gupta, "The SPLASH-2 Programs: Characterization and Methodological Considerations," *Proc. 22th Int'l Symp. Computer Architecture (ISCA)*, pp. 24-36, 1995.
- [30] J. Zebchuk and A. Moshovos, "RegionTracker: A Case for Dual-Grain Tracking in the Memory System," technical report, Computer Group, Univ. of Toronto, 2006.



Jeongseob Ahn received the BS degree in computer science and engineering from Dongguk University in 2009 and the MS degree in computer science from KAIST in 2011. He is currently working toward the PhD degree in computer science at KAIST. His research interests focus on computer architecture, operating systems, and virtualization.



Daehoon Kim received the BS degree from the Computer Science Department, Yonsei University, and the MS degree in computer science from the Korea Institute of Science and Technology (KAIST) in 2010. He is currently working toward the PhD degree at the Computer Science Department, KAIST. His research interests are in multicore architecture, cache architecture, and virtualization.



Jaehong Kim received the BS degree in computer engineering from Sung Kyun Kwan University in 2008, and the MS degrees in computer science in 2010 from Korea Advanced Institute of Science and Technology (KAIST). He is currently working toward the PhD degree at KAIST. His research interests include flash memory, cloud computing, and virtualization.



Jaehyuk Huh received the BS degree in computer science from Seoul National University, and the MS and PhD degrees in computer science from the University of Texas at Austin. He is an associate professor of computer science at Korea Advanced Institute of Science and Technology (KAIST). His research interests are in computer architecture, parallel computing, virtualization, and system security. He is a member of the IEEE and the IEEE Computer Society.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.